
FOAM: Factored One-sided Adam-Moment for Practical and Scalable SOAP

Anonymous Author(s)

Affiliation

Address

email

Abstract

Second-order optimizers (*e.g.*, Shampoo, SOAP) have emerged as compelling alternatives to Adam/AdamW and promise faster per-iteration convergence than AdamW in transformer training. However, they still remain costly in practice: even SOAP still pays substantial optimizer-state memory and eigendecomposition overhead. We ask whether the benefits of SOAP can be retained at nearly the efficiency of AdamW. In this paper, we answer yes with a lightweight yet improved SOAP, Factored One-sided Adam-Moment (FOAM). The path to practical impact is twofold: first, FOAM employs input-only preconditioning, dropping the output-side Kronecker factor to reduce memory and remove one eigendecomposition. A per-row Hessian decomposition identifies this as the right one-sided choice when output curvature is approximately isotropic. Second, FOAM replaces SOAP’s dense rotated second moment with a factored Adam-moment estimator based on row-by-column statistics. Under the K-FAC Fisher assumption, this estimator is further proven to be pointwise equivalent to SOAP’s update. Our empirical results support the claim that FOAM matches strong SOAP and Muon baselines in validation loss across a diverse scale of large language model (LLM) pre-training, uses optimizer state closer to AdamW, runs within Muon’s wall time, and retains robustness across a five-fold learning-rate range. Scaling to 760M parameters, FOAM demonstrates that SOAP’s gains need not require SOAP’s cost.

1 Introduction

Adam [17] and AdamW [22] have long been the default for pre-training transformer architectures, including large language models (LLMs). Their successors, including memory-efficient adaptive optimizers [35–37, 13], show that comparable optimization quality can be achieved at reduced computational cost. In another direction, Shampoo [14], a foundational method in the family of Kronecker-factored second-order optimizers, has shown that AdamW’s optimization quality can be surpassed. Its most practical variant, SOAP [33], the Adam-in-eigenbasis version of Shampoo, reports 35–40% improvements in wall-clock time and iteration count over AdamW at large batch sizes, building upon the Distributed Shampoo baselines [1, 30] that won the 2023 AlgoPerf benchmark [4].

Still, practitioners often default to AdamW or Muon optimizer [16], because SOAP achieves its gains by sacrificing computational efficiency that requires roughly $3\times$ larger optimizer state and significantly slower per-step wall-clock time. More specifically, SOAP’s required computational cost is dominated by computing the components: the first moment, the second moment in the eigenbasis-rotated space, and the two Kronecker factor matrices $L = G^\top G$ (n^2) and $R = GG^\top$ (m^2). Its runtime is further occupied by the periodic eigendecomposition of these factor matrices, which makes SOAP substantially more expensive than AdamW despite its strong optimization performance.

This paper aims to retain SOAP-level quality while approaching AdamW-level cost. Our motivation is simple: if SOAP’s computational overhead can be sufficiently reduced, its optimization advantages

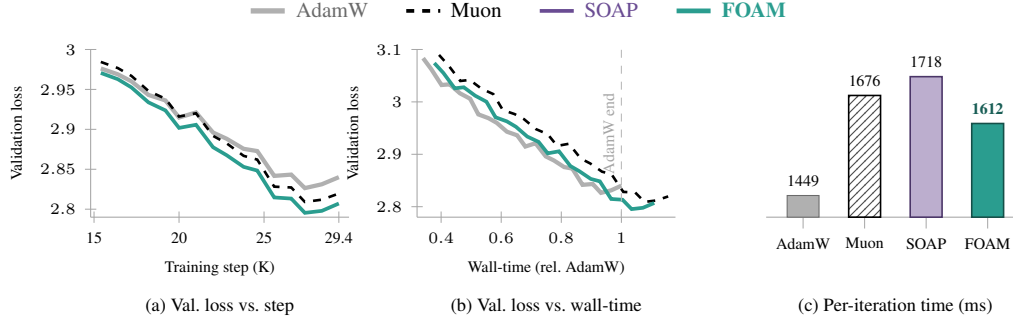


Figure 1: **FOAM is a Pareto-efficient second-order optimizer.** GPT-2 large (760M), 29.4K steps, each optimizer at its tuned learning rate. **(a) Per-step.** Validation loss against training step (zoomed to the second half). With matched step counts, FOAM dominates throughout. **(b) Wall-clock.** Same runs vs. wall time, normalized so AdamW finishes at $1.0\times$ (Muon and FOAM finish at $1.16\times$ and $1.11\times$ due to per-step overhead). Within AdamW’s budget, FOAM already matches AdamW’s final loss; given its full budget, it attains the lowest loss of all three. **(c) Per-iteration cost.** Vanilla SOAP (full eigendecomposition) is slowest; FOAM’s in-only one-sided factorization cuts $\sim 6\%$ off SOAP and is also faster than Muon. Together, FOAM inherits SOAP-class per-step quality at near-AdamW per-step cost.

could make it a practical successor to AdamW. To this end, we revisit SOAP’s major cost components and show that several can be relaxed or replaced without sacrificing performance. While the original SOAP paper explored variants beyond the base formulation, subsequent works [CITE] likewise sought to improve SOAP; our approach addresses the problem along different structural axes.

Concretely, our approach built on two structural simplifications: input-only preconditioning and factored Adam-Moment update. The first simplification adopts input-only preconditioning, dropping the output-side factor R and retaining only L , thereby eliminating the m^2 memory cost of R and one of the two eigen-decompositions. The structural justification follows from a per-row Hessian decomposition: output rows of an affine layer share the same input-side outer product $xx^\top$, while transformer Hessians are empirically near-block-diagonal across output rows [19], making the input-only form the natural one-sided approximation. Together, these simplifications reduce SOAP’s optimizer state size from $2mn + 2m^2 + 2n^2$ to $mn + 2n^2 + m + n$, while reducing the amortized eigendecomposition count from two to one. We call the resulting method **FOAM** (**F**actored **O**ne-sided **A**dam-**M**oment). Next, the factored Adam-Moment update replaces the dense rotated second moment with a row-and-column factored estimator, $r \in \mathbb{R}^m$ and $c \in \mathbb{R}^n$, with trace normalization, reducing the buffer size from mn to $m + n$. We show that under the K-FAC Fisher assumption [25], this estimator is pointwise equivalent to the SOAP update.

Finally, we present Figure 1, which highlights FOAM’s main advantages. On pre-training, FOAM matches the strong Muon and SOAP baselines on validation loss, uses optimizer state within 50% of SOAP, runs within 70% of Muon’s per-step wall-time, and matches AdamW’s hyperparameter robustness over a five-fold learning-rate range. The advantage holds from 124M to 760M parameters under μP [34] learning-rate transfer. In summary, our contributions are: (i) a structural justification of input-only preconditioning, with a per-row Hessian decomposition for the theoretical backbone and empirical verifications with LLMs with recent architectural elements; (ii) a factored Adam-Moment update with trace normalization, proved pointwise equivalent to SOAP under K-FAC and empirically validated through KFAC-residual measurements during training; and (iii) the resulting algorithm FOAM, a scalable and practical SOAP variant that delivers SOAP-class quality with AdamW-class memory and Muon-class per-step wall-time across GPT-2 from 124M to 760M.

2 Related Work

Adaptive optimizers can be categorized by how aggressively they approximate the empirical Fisher $F = \mathbb{E}[\text{vec}(G) \text{vec}(G)^\top]$, which all of Adam [17]/AdamW [22], Shampoo [14], and SOAP [23] implicitly use as a curvature surrogate in place of the true Fisher [18].

2.1 Empirical Fisher and Kronecker-factored Optimizers

AdaGrad [6] introduced per-coordinate scaling by an empirical second-moment statistic, an early diagonal approximation of the inverse empirical Fisher. K-FAC [25] replaces the diagonal with a Kronecker product, $F \approx \mathbb{E}[aa^\top] \otimes \mathbb{E}[ss^\top]$ where a is the layer input and s the post-activation gradient, the assumption under which our factored Adam-Moment update is exact. Eschenhagen et al. [8] adapt K-FAC to transformer architectures via the *expand* and *reduce* flavors. Shampoo [14] carries this idea to the matrix parameters: it maintains EMAs of $L = G^\top G$ and $R = GG^\top$ and steps in $L^{-1/4}GR^{-1/4}$, which Morwani et al. [26] show is exactly the square root of the Kronecker-factored empirical Fisher. SOAP [33] eigendecomposes L and R and runs Adam in the resulting rotated coordinates, an EKFAC-style construction [12] that Lu et al. [23] interpret as gradient whitening. Variants modify specific pieces: CASPR [7] replaces the $1/4$ -power with a diagonal post-rotation correction; KL-Shampoo / KL-SOAP [20] recasts the family via KL-divergence minimization; Purifying Shampoo [9] introduces trace-style normalization heuristics that corroborate our trace-norm choice in §4.2. The 1-D Shampoo input-only fallback in distributed Shampoo [1] drops R on memory grounds; we strengthen it with a structural argument (§4.1). Purifying Shampoo [9] introduces an adaptive tolerance criterion for skipping the periodic eigendecomposition; we use this rule as our amortized policy.

2.2 More-approximate Optimizers

A second line of work adopts coarser approximations to the empirical Fisher in exchange for lower computational cost. Adam [17, 22] keeps only the diagonal D_{mn} , sacrificing all matrix structure. Signum [2] keeps only the sign of the (momentum-smoothed) gradient. Muon [16] replaces the second-moment buffer with on-the-fly Newton-Schulz orthogonalization $(MM^\top)^{-1/2}M$, which Ma et al. [24] show can be derived from the structured Fisher assumption $F = I_n \otimes \tilde{H}$. AdaMuon [31] adds Adam-style adaptation in the orthogonalized basis, the closest cousin to FOAM in this branch; we treat Muon and AdaMuon as the strong non-SOAP baselines. SPlus [11] pursues a related “memory-efficient SOAP cousin” axis via Lookahead and sign normalization; we include it as a baseline. Recent Muon variants (NorMuon, Muon², NAMO, Polar Express) refine specific components and are represented by AdaMuon. A parallel branch targets memory-efficient AdamW rather than matching SOAP-class quality: Adam-mini [35], GaLore [36], APOLLO [37], Alice [13], and SWAN [24]; we exclude these methods from the main Comparison since their stated goal is matching Adam-level performance at lower computational costs, whereas our goal is to attain SOAP-level performance at near-Adam cost.

Positioning our method. FOAM lies at the intersection of these two branches: it stays within the Kronecker-factored Fisher framework but borrows the factored-second-moment idea from Adafactor [29]’s row-by-column estimator, applied here in the rotated coordinates of SOAP. Based on SOAP, we replace the two factor matrices with a single input-only factor, replace the dense rotated second moment with the factored estimator.

3 Preliminary: Shampoo and SOAP

We consider a transformer with matrix parameters $W \in \mathbb{R}^{m \times n}$, where n and m denote the input and output dimensions, respectively. Let $G_t \in \mathbb{R}^{m \times n}$ denote the mini-batch gradient at step t . Following the convention of SOAP [33], we define the two Kronecker factors L_t and R_t as running second-moment estimator:

$$L_t = \mathbb{E}[G_t G_t^\top] \in \mathbb{R}^{m \times m}, \quad R_t = \mathbb{E}[G_t^\top G_t] \in \mathbb{R}^{n \times n}, \quad (1)$$

L_t acts on the left side (**output**) of G_t , and R_t acts on the right side (**input**). Shampoo [14] takes a fractional inverse of each factor and applies the preconditioned update:

$$W_{t+1} = W_t - \eta L_t^{-1/4} G_t R_t^{-1/4}, \quad (2)$$

with learning rate η . SOAP [33] instead eigen-decomposes L_t and R_t to obtain orthogonal rotations $Q_L \in \mathbb{R}^{m \times m}$ and $Q_R \in \mathbb{R}^{n \times n}$, and runs Adam in the rotated coordinates:

$$G_{\text{rot}} = Q_L^\top G Q_R \in \mathbb{R}^{m \times n}, \quad (3)$$

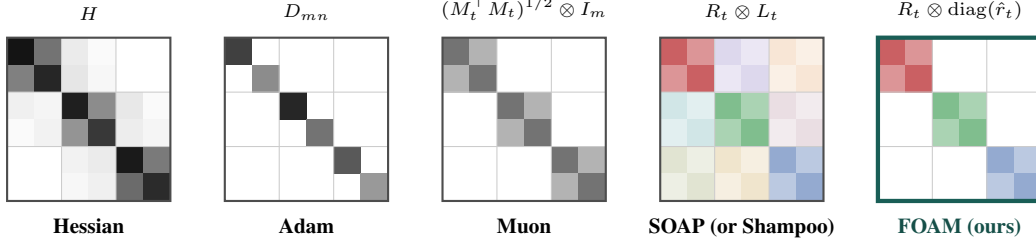


Figure 2: **Hessian and its approximation across optimizers.** For $W \in \mathbb{R}^{m \times n}$ with $m=3, n=2$, we visualize the $mn \times mn$ Hessian alongside the preconditioner of each optimizer; colored cells are non-zero, white cells zero, and shading reflects relative magnitude. The Hessian is near-block-diagonal. **Adam** (D_{mn}): fully diagonal approximation with no Kronecker structure. **Muon** $((M_t^T M_t)^{1/2} \otimes I_m)$: m identical outer-diagonal blocks, built statelessly from the current momentum via $M_t(M_t^T M_t)^{+1/2}$. **SOAP** ($R_t \otimes L_t$): full Kronecker fit with EMAs on both sides; all m^2 outer blocks populated. **FOAM** ($R_t \otimes \text{diag}(\hat{r}_t)$, ours): replaces Muon’s identity output with a per-block diagonal scaling, and its stateless input factor with the EMA of $G_t^T G_t$.

Algorithm 1 FOAM update at step t for $W \in \mathbb{R}^{m \times n}$. $\mathbf{1}_d$ is the all-ones vector in $\mathbb{R}^d$, $G^{\circ 2}$ denotes element-wise square, $\oslash$ denotes element-wise division, and $\text{mean}(\cdot)$ denotes the arithmetic mean of a vector’s entries.

Require: gradient G ; hyperparameters $(\eta, \lambda, \beta_1, \beta_2, \beta_{\text{sh}}, \varepsilon, \text{pf})$; state M, r, c, R, Q_R

- 1: $G_{\text{rot}} \leftarrow G Q_R$ (rotate input axis)
- 2: $M \leftarrow \beta_1 M + (1 - \beta_1) G_{\text{rot}}$ (first moment)
- 3: $v^{\text{row}} \leftarrow \beta_2 r + (1 - \beta_2) G_{\text{rot}}^{\circ 2} \mathbf{1}_n / n$, $v^{\text{col}} \leftarrow \beta_2 c + (1 - \beta_2) \mathbf{1}_m^T G_{\text{rot}}^{\circ 2} / m$ (factored second moments)
- 4: $\hat{M} \leftarrow M / (1 - \beta_1^t)$, $\hat{v}^{\text{row}} \leftarrow v^{\text{row}} / (1 - \beta_2^t)$, $\hat{v}^{\text{col}} \leftarrow v^{\text{col}} / (1 - \beta_2^t)$ (bias correction)
- 5: $\tau \leftarrow \text{tr}(R) / (n \cdot \text{mean}(\hat{r}) \cdot \text{mean}(\hat{c}))$, $\hat{v}_{\text{rot}} \leftarrow \tau \hat{v}^{\text{row}} \hat{v}^{\text{col}T}$ (reconstruct second moment)
- 6: $W \leftarrow W - \eta (\hat{M} \oslash \sqrt{\hat{v}_{\text{rot}} + \varepsilon}) Q_R^T - \eta \lambda W$
- 7: **if** $t \bmod \text{pf} = 0$ **then**
- 8: $R \leftarrow \beta_{\text{sh}} R + (1 - \beta_{\text{sh}}) G^T G$, and update Q_R if R has drifted from diagonal in the current basis (§4.2)
- 9: **end if**

116 following the EKFAC tradition [12] that runs Adam in the eigen-basis of a Kronecker-factored
 117 preconditioner. The per-step update maintains a dense second moment $v_{\text{rot}} \in \mathbb{R}^{m \times n}$ in the rotated
 118 space. The rotations Q_L and Q_R are refreshed every pf (precondition frequency) steps via two
 119 eigen-decompositions of L_t and R_t . L_t and R_t require $m^2 + n^2$ memory, the rotated second
 120 moment v_{rot} requires an additional mn memory, and the periodic eigen-decompositions incur further
 121 computational overhead. These costs constitute the overhead structure that FOAM addresses.

122 4 Factored One-sided Adam-Moment (FOAM)

123 We propose **FOAM**, **F**actored **O**ne-sided **A**dam-**M**oment, which combines two structural simplifica-
 124 tions of SOAP into a single optimizer. FOAM applies an input-side K-FAC factor combined with
 125 a factored output diagonal: it maintains only the input-side factor R_t , runs Adam in the eigenbasis
 126 $Q_R = \text{eigh}(R_t)$, and replaces SOAP’s dense rotated second moment $v_{\text{rot}} \in \mathbb{R}^{m \times n}$ with a row-
 127 column factored estimator $\hat{v}_{\text{rot}} = \tau \cdot \hat{v}^{\text{row}} \hat{v}^{\text{col}T}$. Here $\hat{v}^{\text{row}} \in \mathbb{R}^m$ and $\hat{v}^{\text{col}} \in \mathbb{R}^n$ are bias-corrected
 128 EMAs derived from the rotated gradient G_{rot} , and The two simplifications together give FOAM
 129 the implicit Fisher $R_t \otimes \text{diag}(\hat{r}_t)$, a full input-side factor with a diagonal output side, derived in
 130 Algorithm 1 and the following sections. Figure 2 compares this preconditioner structure with those
 131 of Adam, Muon, and SOAP against the near-block-diagonal Hessian of a transformer layer.

132 4.1 Input-only Preconditioning

133 Our idea is to entirely remove the output-side factor L_t entirely and maintain only $R_t \leftarrow \beta_{\text{sh}} R_{t-1} +$
 134 $(1 - \beta_{\text{sh}}) G_t^T G_t$ with its eigen-basis $Q_R = \text{eigh}(R_t)$. The gradient rotation is applied only for the

input axis, i.e., $G_{\text{rot}} = G Q_R \in \mathbb{R}^{m \times n}$. This leads to reduced computational costs: memory required for the factor state drops from $\mathcal{O}(m^2 + n^2)$ to $\mathcal{O}(n^2)$, and the number of eigen-decompositions per amortized step drops from two to one¹.

A conceptual justification comes from the curvature of an affine layer with cross-entropy loss, which is conventionally used in LLM training or classification tasks. For $y = Wx$ and $p = \text{softmax}(y)$, Hessian of loss $\ell = -\log p_t$ for W decomposes exactly as

$$H = xx^\top \otimes (\text{diag}(p) - pp^\top). \quad (4)$$

The output-side factor $\text{diag}(p) - pp^\top$ has diagonal entries $p_i(1 - p_i)$ and off-diagonal entries $-p_i p_k$. As training sharpens the predictive distribution toward the one-hot target, the off-diagonal entries $p_i p_k$ vanish faster than the diagonal entries $p_i(1 - p_i)$, an observation traced from Collobert [3] through Kunstner et al. [19]. The output-side curvature thus becomes increasingly diagonal during training, and after taking expectation over a batch:

$$\mathbb{E}[H] \approx \mathbb{E}[xx^\top] \otimes \text{diag}(c), \quad c_i = \mathbb{E}[p_i(1 - p_i)]. \quad (5)$$

This matches the implicit Fisher of FOAM exactly, with the input-side curvature $\mathbb{E}[xx^\top]$ tracked by R_t and the per-output curvature scale c_i absorbed into $\hat{r}_t$. The output side need not be homogeneous across rows; FOAM only requires that it is dominated by its diagonal, which the cross-entropy vanishing mechanism naturally provides as training progresses.

Beyond the affine setting. The vanishing-off-diagonal mechanism is specific to the affine derivation, but the resulting near-block-diagonal Hessian structure has been observed to extend to deep transformers throughout training [19]. We provide additional indirect evidence specific to our setting through Gauss-Newton alignment measurements in §5.4.

4.2 Factored Eigenvalue Under Input-only Preconditioning

Even with the input-only simplification, SOAP still has overheads due to the dense rotated second-moment buffer $v_{\text{rot}} \in \mathbb{R}^{m \times n}$. Our next step is to replace it with two factored estimators $v^{\text{row}} \in \mathbb{R}^m$ and $v^{\text{col}} \in \mathbb{R}^n$, updated as the row- and column-mean of $G_{\text{rot}}^{\circ 2} = G_{\text{rot}} \circ G_{\text{rot}}$, and reconstruct the rotated second moment as an outer product with trace normalization:

$$\hat{v}_{\text{rot}}[i, j] = \tau \cdot \hat{v}^{\text{row}}[i] \cdot \hat{v}^{\text{col}}[j], \quad \tau = \frac{\text{tr}(R) / n}{\text{mean}(\hat{v}^{\text{row}}) \cdot \text{mean}(\hat{v}^{\text{col}})}. \quad (6)$$

The state shrinks from mn to $m + n$ per matrix parameter. The trace-matching scalar τ is chosen so that $\text{mean}(\hat{v}_{\text{rot}}) = \text{tr}(R)/n$, fixing the overall scale of the reconstructed second moment to that of the input-side factor. We now justify why this rank-one factorization is sufficient.

Lemma 1 (Covariance of the input-rotated gradient). *Let $g = \text{vec}(G) \in \mathbb{R}^{mn}$ and assume the K -FAC factorization $\mathbb{E}[gg^\top] = R \otimes L$. Let $g' = \text{vec}(G Q_R)$ be the input-rotated gradient in column-major form. Then*

$$\mathbb{E}[g' g'^\top] = \Lambda_R \otimes L, \quad (7)$$

where $\Lambda_R = Q_R^\top R Q_R$ is the diagonal eigenvalue matrix of R .

Proof. Using the column-major identity $\text{vec}(AGB) = (B^\top \otimes A) \text{vec}(G)$ with $A = I_m$ and $B = Q_R$, we have $g' = (Q_R^\top \otimes I_m) g$. Therefore,

$$\mathbb{E}[g' g'^\top] = (Q_R^\top \otimes I_m) \mathbb{E}[g g^\top] (Q_R \otimes I_m) = (Q_R^\top \otimes I_m) (R \otimes L) (Q_R \otimes I_m). \quad (8)$$

The Kronecker mixed-product identity $(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$, applied twice, gives

$$(Q_R^\top \otimes I_m) (R \otimes L) (Q_R \otimes I_m) = (Q_R^\top R Q_R) \otimes (I_m L I_m) = \Lambda_R \otimes L. \quad (9)$$

□

¹ SOAP [23] explored a one-sided variant that drops the larger-side factor in their efficiency-oriented ablations. In contrast, our method explicitly removes the output-side factor and uses the input-side only. Furthermore, Frans et al. [10] tested input- and output-only variants as part of their preconditioning ablations. Our work, to our knowledge, is the first to demonstrate that such a one-sided formulation can replace the original two-sided design as a more practical alternative.

Table 1: **Per-parameter complexity comparison.** Optimizer state size, per-step compute, and amortized eigendecomposition cost for a matrix parameter $W \in \mathbb{R}^{m \times n}$. FOAM matches Muon’s per-step cost while reducing both state and eigendecomposition cost relative to SOAP.

	AdamW	Muon	SOAP	FOAM
Optimizer state	$2mn$	mn	$2mn + 2m^2 + 2n^2$	$mn + 2n^2 + (m + n)$
Per-step cost	$\mathcal{O}(mn)$	$\mathcal{O}(mn^2)$	$\mathcal{O}(mn(m + n))$	$\mathcal{O}(mn^2)$
Amortized (per pf)	–	–	$\mathcal{O}(m^3 + n^3)$	$\mathcal{O}(n^3)$

Lemma 1 tells us two things about the rotated gradient. First, the input-side covariance becomes diagonal: rotating by Q_R decorrelates the input directions and replaces R with its eigenvalues Λ_R . Second, the output-side factor L is unchanged. By §4.1, L is itself driven toward its diagonal by the cross-entropy mechanism, so the rotated covariance reduces to $\Lambda_R \otimes \text{diag}(L)$, a fully diagonal matrix whose entries take the rank-one form:

$$\mathbb{E}[(GQ_R)[i, j]^2] = \lambda_R[j] \cdot L[i, i], \quad (10)$$

with $\lambda_R[j]$ the j -th diagonal entry of Λ_R . The right-hand side is a per-row factor depending only on i times a per-column factor depending only on j , so the entire $m \times n$ second-moment surface is captured by $m + n$ numbers rather than mn . The row- and column-mean of $G_{\text{rot}}^{\circ 2}$ are the natural estimators of these two factors, up to a global scalar that the trace normalization τ removes, so $\tau \cdot \hat{v}^{\text{row}} \hat{v}^{\text{col}\top}$ recovers the same diagonal preconditioner that SOAP’s dense mn buffer would, with no information loss when the K-FAC and diagonal- L assumptions hold.

Amortized eigenbasis update. The eigenbasis Q_R is refreshed every pf steps via the adaptive skip rule of Purifying Shampoo [9]: form $B = Q_R^\top R_t Q_R$ and skip the eigen-decomposition whenever $\|B - \text{diag}(B)\|_F / \|B\|_F \leq \text{tol}$, keeping the previous Q_R . The tolerance tol is a small hyperparameter; an eigen-decomposition is invoked only when R_t has drifted noticeably from diagonal in the current basis, which becomes infrequent late in training as the basis stabilizes.

Complexity. Table 1 compares per-parameter optimizer state and compute. SOAP requires state $2mn + 2m^2 + 2n^2$ (first and rotated second moments, two factor EMAs, two eigenbases). FOAM reduces this to $mn + 2n^2 + (m + n)$: a single first-moment buffer, the row-column estimators r, c , and the input-side factor R with its eigenbasis Q_R . Per-step cost is $\mathcal{O}(mn^2)$ from the rotation GQ_R , matching Muon, and the amortized eigen-decomposition cost drops from $\mathcal{O}(m^3 + n^3)$ to $\mathcal{O}(n^3)$.

5 Experiment

This section empirically answers two questions in turn: **Q1:** Does FOAM match the strong Muon and SOAP baselines on transformer pre-training across model sizes (§5.2)? **Q2:** individual design components of FOAM well-founded and empirically contributive (§5.4)?

5.1 Experimental Setup

Setup. We pre-train the models following a modern LLM architectural setup (RoPE [32], RMSNorm, untied embeddings, MLP with GELU, no bias) at three scales following GPT-2 [28]: small (124M), medium (350M), and large (760M), on FineWeb [27] at a $1 \times$ Chinchilla compute budget (≈ 20 tokens per parameter) [15], giving 2.49B / 7.10B / 15.49B training tokens and 4,750 / 13,550 / 29,550 optimizer steps, respectively at a fixed effective batch size of 0.5M tokens (1k sequence length). All runs use bfloat16 mixed precision on $8 \times \text{H100}$ (FSDP at medium/large), with cosine learning-rate decay to 10% of the peak after 2% warmup, gradient clipping at 1.0, and $\epsilon = 10^{-8}$ for Adam.

Three-group optimizer routing. Following the academic-setup convention of Vyas et al. [33], Frans et al. [11], Jordan et al. [16], only the matrix parameters (Q/K/V/O attention projections and MLP linears) are preconditioned by the optimizer under study; embeddings, the lm-head, and 1D / norm parameters are routed to a fixed AdamW group with LR 10^{-2} , $\beta = (0.95, 0.95)$, no weight decay on embeddings, and weight decay 10^{-3} on the lm-head. This isolates the algorithmic comparison to the matrix-parameter group and makes per-method optimizer-state and step-time numbers directly comparable. Unless noted, all groups use $\beta = (0.95, 0.95)$, and SOAP variants use precondition frequency 10 with E-Shampoo skip tolerance 0.1 [9].

Table 2: **Pre-training results.** For each method and scale at its per-size best LR, we report validation loss (*val*), matrix-parameter optimizer state in GiB (*state_{mat}*, *state*), median per-iteration optimizer-step time in ms (*step*), and total training wall-clock in hours (*wall*) on 8×H100 (FSDP at medium/large). Small is 3-seed mean $\pm$ std at the inner cell of a 5×5 LR/WD grid; medium/large are single-seed at the μ P-anchored inner LR. SOAP is the bilateral dense-*v* variant, FOAM the input-only + factored counterpart. *state_{mat}* excludes the embed/lm-head/norm AdamW group (measured at small, analytical fp32 lower bound at medium/large; the bound is within $\pm 15\%$ of the small measurement). *wall* at small is profile-clean ($\text{iter}_{\text{ms}} \times \text{steps}$ on a single pod); at medium/large it is end-to-end including eval/ckpt overhead. SOAP is the bilateral, dense-*v* variant; FOAM is the input-only + factored counterpart. Bold marks the best validation loss per size.

Optimizer	small (124M)				medium (350M)				large (760M)			
	<i>val</i>	<i>state</i>	<i>step</i>	<i>wall</i>	<i>val</i>	<i>state</i>	<i>step</i>	<i>wall</i>	<i>val</i>	<i>state</i>	<i>step</i>	<i>wall</i>
AdamW	3.362 $\pm$.009	0.63	1.3	0.38	3.044	2.25	2.7	2.68	2.832	5.27	1.5	11.88
Muon	3.304 $\pm$.010	0.32	69.7	0.48	3.011	1.13	129.5	3.19	2.802	2.64	230.4	13.73
AdaMuon	3.300 $\pm$.008	0.32	72.9	0.51	3.002	1.13	138.7	3.21	2.798	2.64	247.9	13.95
SPlus	3.325 $\pm$.007	2.85	43.7	0.48	—	—	—	—	—	—	—	—
SOAP	3.295$\pm$.011	2.85	53.5	0.48	2.999	10.13	104.2	3.20	—	—	—	—
FOAM (ours)	3.304 $\pm$.007	1.42	46.8	0.46	2.998	5.06	93.5	3.12	2.786	11.87	163.1	13.18

Hyperparameter search. Following the matrix-whitening convention of Frans et al. [11], each method is tuned to a ± 0.005 validation-loss resolution. At small we run a 5×5 LR/WD grid (factor $10^{1/8} \approx 1.33$ on LR, $10^{1/4} \approx 1.78$ on WD) per optimizer at $\beta = (0.95, 0.95)$, re-centering on grid edges, and re-validate the inner cell with a 3×3 multi-seed sweep over seeds {42, 1339, 2024}. For medium and large, dense LR/WD grids are infeasible: we transfer the small inner LR via a μ P [34] ($\propto 1/\text{width}$) anchor, sweep 5 LR points (factor $10^{1/8}$) around it with WD held at the small inner optimum, and report the inner cell as the per-size point estimate. Validation loss is reported as 3-seed mean $\pm$ std at small and as a single-seed value at medium / large.

Baselines. The headline comparison spans AdamW [22], Muon [16], AdaMuon [31], SOAP [33] (the bilateral, dense-*v* variant), and FOAM (ours) at all three scales. At GPT-2 small we additionally evaluate, SPlus [11], and the SOAP single-side variants used in our ablation (§5.4).

5.2 Main Results

Table 2 reports our headline metrics across three model scales. FOAM matches SOAP within 1σ at small (3.304 $\pm$.007 vs. 3.295 $\pm$.011) and is statistically tied at medium ($\Delta = 0.0002$, well within the ~ 0.008 seed std observed at small); at large, FOAM achieves the best validation loss of the four headline methods (2.786 vs. AdaMuon 2.798, SOAP 2.808, AdamW 2.832). The cost gap to SOAP *widens at scale*. On matrix-parameter state, FOAM is $\sim 2.3\times$ smaller than SOAP at every size (1.42 vs. 2.85 GiB at small, 4.69 vs. 10.88 at medium, 10.99 vs. 25.49 at large). On wall-time, FOAM stays at 1.19×AdamW across all three sizes, while SOAP grows from 1.24× at small to 1.86× at medium and 2.02× at large as the per-step eigh cost becomes quadratic in width. Per-iteration step time at small is reported in Table 3. *state_{mat}* excludes the embed/lm-head/norm AdamW group (measured at small, analytical fp32 lower bound within $\pm 15\%$ at medium/large); *wall* is profile-clean at small and end-to-end (including eval/ckpt) at medium/large. FOAM is the only optimizer in the comparison that reaches SOAP-class final loss while remaining close to AdamW on both state and wall-time.

5.3 Hyperparameter robustness

A practical optimizer must also tolerate imprecise tuning. Figure 3 sweeps LR, WD, and β_1, β_2 around each optimizer’s inner best on GPT-2 small. FOAM’s bowl shape and width track AdamW and SOAP across all four axes within 0.05 nats. The one notable asymmetry is AdamW’s catastrophic collapse at $\beta_1 = 0.99$ (val ≈ 3.85 , off-scale) where SOAP and FOAM degrade only mildly ($+0.01$ – 0.02 nats); FOAM thus inherits SOAP’s wider safe range on the momentum axis.

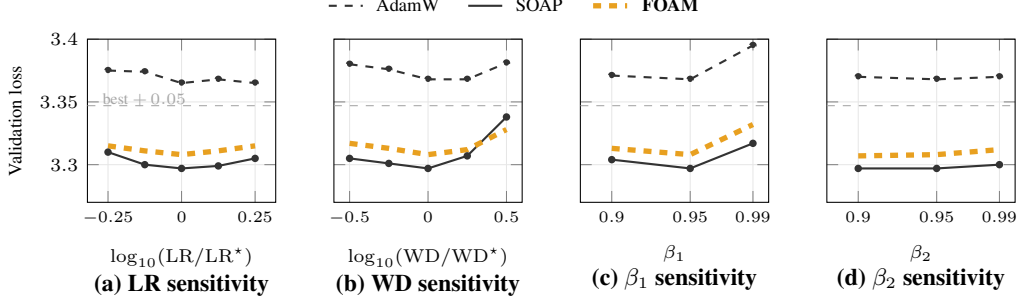


Figure 3: **Hyperparameter robustness around each optimizer’s best cell on GPT-2 small.** For AdamW, SOAP, and FOAM we sweep 5 LR points (factor $10^{1/8}$), 5 WD points (factor $10^{1/4}$) at the best LR, and 3 values each of β_1 and β_2 at the best LR/WD; all other HPs are held fixed. The dashed gray line marks SOAP’s “best + 0.05 nats” stability threshold (3.347).

Table 3: **FOAM-construction ablation on GPT-2 small (124M).** Cross of preconditioning side (Q_L, Q_R retained) and factorization of the second-moment v , on top of SOAP. Each variant is reported at its own best 5×5 LR/WD cell as 3-seed mean $\pm$ std. state is the matrix-parameter optimizer state (GiB), step is the median per-iteration optimizer-step time (ms), and wall is the total training wall-clock on a single $8 \times H100$ pod.

Precond.	Factored v	val loss	state (GiB)	step (ms)	wall (h)
both		$3.295 \pm .011$	2.92	53.5	0.48
both	✓	$3.293 \pm .008$	2.59	67.1	0.49
input		$3.302 \pm .008$	1.78	46.6	0.44
output		$3.322 \pm .010$	1.78	40.9	0.46
input	✓	$3.304 \pm .007$	1.46	46.8	0.46

241 5.4 Analysis

242 We first measure what each FOAM simplification contributes to quality and cost (§5.4), then ask
 243 whether input-only preconditioning is structurally justified on actual GPT-2 layers (§5.4), and finally
 244 whether the factored update stays close to the dense SOAP update in practice (§5.4).

245 **FOAM-construction ablation.** Table 3 walks from SOAP to FOAM by toggling two structural
 246 simplifications independently: input-only preconditioning (drop the output-side rotation Q_L) and
 247 the factored second moment (replace dense v with row/column estimators r, c). Each variant is
 248 evaluated at its own best HP cell with 3 seeds. Input-only preconditioning reduces both optimizer
 249 state and step time substantially while preserving validation loss within ~ 0.01 nats of full SOAP,
 250 whereas the symmetric counterpoint, output-only preconditioning, costs the same matrix-state as
 251 input-only but loses ~ 0.02 nats in validation, foreshadowing the structural asymmetry diagnosed
 252 next. Factorization further trades a small step-time increase for a meaningful reduction in optimizer
 253 state with no measurable loss in validation. Stacked, the two simplifications yield FOAM, which
 254 retains SOAP’s validation while halving the matrix-parameter optimizer state.

255 **Gauss-Newton fit.** Figure 4 compares six implicit preconditioners against the per-layer Gauss-
 256 Newton matrix H_{GN} on the 72 matrix parameters of the 12-layer GPT-2 small backbone. SOAP’s
 257 full bilateral preconditioner (~ 0.51 – 0.60) is the closest Kronecker approximant of H_{GN} , $\sim 6\times$ above
 258 Adam-diag and $\sim 2\times$ above Muon. The asymmetry between input and output rotations is sharp and
 259 persistent: dropping the output-side rotation costs only $\sim 25\%$ of the alignment (input-only retains
 260 $\sim 75\%$ of SOAP-full at ~ 0.38), whereas dropping the input-side rotation costs $\sim 80\%$ (output-only
 261 retains only $\sim 20\%$ at ~ 0.10). FOAM (input-only + factored) tracks SOAP-input-only within 1–2%
 262 at every checkpoint, indicating that factoring the eigenvalue tensor preserves nearly all of input-only’s
 263 curvature alignment. This asymmetry, in which the input-feature axis carries the dominant curvature
 264 structure, is the empirical justification for the input-only recipe used in the main results.

265 **Factored update validity.** Figure 5 reports two diagnostics that explain why the factored variants
 266 of Table 3 preserve quality. The rank-1 share in panel (b) is the load-bearing signal: in Adam’s raw
 267 g^2 basis V is only ~ 0.54 – 0.63 rank-1 separable and drifts down during training, while in SOAP’s
 268 eigenbasis the rotated V remains at ~ 0.90 (full bilateral) or ~ 0.84 (input-only) throughout. The

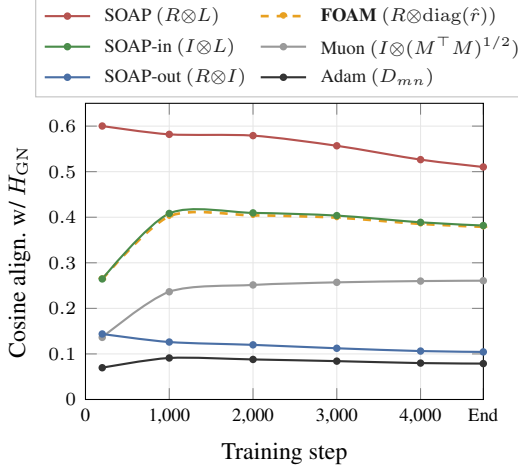


Figure 4: **Layer-wise GaussNewton alignment.** For each of 72 matrix parameters, we estimate $H_{GN} = J_\ell^T G_y J_\ell$ via 30 Hutchinson probes (shared across variants) with the closed-form softmax-CE output Hessian, and compute the Frobenius cosine $\cos(P, H_{GN})$ in sqrt-power form for six implicit preconditioners. Curves report the per-variant mean over the 72 block matrices.

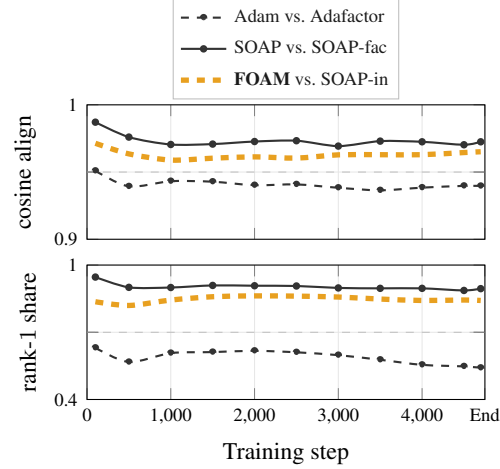


Figure 5: **Validity of the factored second moment.** On the dense run of each regime, we form the rank-1 approximation $V_{fac} = (L \otimes R) / \sum L$ from the row and column sums L, R of the second-moment estimate V , and report (a) the Frobenius cosine between V_{fac} and V and (b) the rank-1 share $\sigma_1(V) / \sum_i \sigma_i(V)$, averaged across all matrix parameters.

cosine in panel (a) follows accordingly: ~ 0.97 in SOAP’s full basis, ~ 0.96 in input-only (= FOAM), and only ~ 0.94 in Adam’s raw basis. Consequently, factoring the SOAP-rotated V loses essentially nothing (val penalty < 0.005 nats), whereas factoring Adam’s V degrades quality more visibly (Table 3, 29). Combined with the input-only H_{GN} alignment of Figure 4, this completes the structural rationale for FOAM: rotate into the input-side eigenbasis, where V becomes nearly rank-1 separable, then factor.

6 Discussion and Conclusion

Limitations and future work. We verify FOAM up to 760M at $1 \times$ Chinchilla; long-horizon training, shard distributed for $\geq 7B$ (sharded A_L with collective rotation), interaction with schedule-free averaging [5], and `lm_head`-specific input-only variants are natural future work. Since we believe our experiments are computationally intensive, resource constraints limit our evaluation to models up to 760M parameters, so scalability beyond 1B remains our future work.

Concurrent works in different brackets. Memory-efficient adaptive optimizers (Adam-mini [35], GaLore [36], APOLLO [37], Alice [13], SWAN [24]) compresses the AdamW state but targets matching AdamW quality at a lower cost rather than beating it. Alternative theoretical framings of factored SOAP-style updates include KL-Shampoo [20] and Purifying Shampoo [9]; the latter independently identifies trace-style normalization as a heuristic that matters, corroborating our trace-norm choice in Section 4.2. Other Muon variants (NorMuon, Muon², NAMO, Polar Express) refine specific components and are represented by AdaMuon in our main comparison. Sophia [21] uses a different curvature signal (diagonal Hessian); Schedule-Free AdamW [5] attacks an orthogonal axis (no LR schedule).

Conclusion. We presented *FOAM*, a scalable and practical SOAP variant built on two structural simplifications: input-only preconditioning, justified by a per-row Hessian decomposition, and a factored Adam-Moment update, proved equivalent to SOAP under K-FAC, combined with E-Shampoo’s adaptive skip rule for the amortized eigenbasis update. Where memory-efficient AdamW variants target Adam-quality at lower cost, FOAM addresses the dual problem and brings SOAP-level quality close to Adam-level cost. FOAM matches the strong Muon and SOAP baselines on LLM pre-training equipped with modern architectural elements while using AdamW-level memory and Muon-level per-step compute, with the gap holding from 124M to 760M. We release our code with a reference implementation and full sweep configurations.

References

- [1] Rohan Anil, Vineet Gupta, Tomer Koren, Kevin Regan, and Yoram Singer. Scalable second order optimization for deep learning. *arXiv preprint arXiv:2002.09018*, 2020.
- [2] Jeremy Bernstein, Yu-Xiang Wang, Kamyar Azizzadenesheli, and Animashree Anandkumar. signSGD: Compressed optimisation for non-convex problems. In *ICML*, 2018.
- [3] Ronan Collobert. Large scale machine learning. In *PhD thesis, Université de Paris VI*, 2004.
- [4] George E Dahl, Frank Schneider, Zachary Nado, Naman Agarwal, Chandramouli Shama Sastry, Philipp Hennig, Sourabh Medapati, Runa Eschenhagen, Priya Kasimbeg, Daniel Suo, et al. Benchmarking neural network training algorithms. *arXiv preprint arXiv:2306.07179*, 2023.
- [5] Aaron Defazio, Xingyu Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled. In *NeurIPS*, 2024.
- [6] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(7), 2011.
- [7] Sai Surya Duvvuri, Fnu Devvrit, Rohan Anil, Cho-Jui Hsieh, and Inderjit S. Dhillon. Combining axes preconditioners through Kronecker approximation for deep learning. In *ICLR*, 2024.
- [8] Runa Eschenhagen, Alexander Immer, Richard E. Turner, Frank Schneider, and Philipp Hennig. Kronecker-factored approximate curvature for modern neural network architectures. In *NeurIPS*, 2023.
- [9] Runa Eschenhagen, Aaron Defazio, Tsung-Hsien Lee, Richard E Turner, and Hao-Jun Michael Shi. Purifying shampoo: Investigating shampoo’s heuristics by decomposing its preconditioner. In *NeurIPS*, 2025.
- [10] Kevin Frans, Pieter Abbeel, and Sergey Levine. What really matters in matrix-whitening optimizers? *arXiv preprint arXiv:2510.25000*, 2025.
- [11] Kevin Frans, Sergey Levine, and Pieter Abbeel. A stable whitening optimizer for efficient neural network training. *arXiv preprint arXiv:2506.07254*, 2025.
- [12] Thomas George, César Laurent, Xavier Bouthillier, Nicolas Ballas, and Pascal Vincent. Fast approximate natural gradient descent in a Kronecker-factored eigenbasis. In *NeurIPS*, 2018.
- [13] Wenbo Gong, Meyer Scetbon, Chao Ma, and Edward Meeds. Towards efficient optimizer design for llm via structured fisher approximation with a low-rank extension. *arXiv preprint arXiv:2502.07752*, 2025.
- [14] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In *ICML*, 2018.
- [15] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, DDL Casas, Lisa Anne Hendricks, Johannes Welbl, and Aidan et al Clark. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 10, 2022.
- [16] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>, 2024. Accessed: 2026-05-07.
- [17] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [18] Frederik Kunstner, Philipp Hennig, and Lukas Balles. Limitations of the empirical Fisher approximation for natural gradient descent. In *NeurIPS*, 2019.
- [19] Frederik Kunstner, Alan Milligan, Robin Yadav, Mark Schmidt, and Alberto Bietti. Heavy-tailed class imbalance and why adam outperforms gradient descent on language models. *Advances in Neural Information Processing Systems*, 37:30106–30148, 2024.
- [20] Wu Lin, Scott C Lowe, Felix Dangel, Runa Eschenhagen, Zikun Xu, and Roger B Grosse. Understanding and improving shampoo and soap via kullback-leibler minimization. *arXiv preprint arXiv:2509.03378*, 2025.
- [21] Hong Liu, Zhiyuan Li, David Leo Wright Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training. In *ICLR*, 2024.
- [22] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

- 347 [23] Yanqing Lu, Letao Wang, and Jinbo Liu. Understanding soap from the perspective of gradient whitening.
348 *arXiv preprint arXiv:2509.22938*, 2025.
- 349 [24] Chao Ma, Wenbo Gong, Meyer Scetbon, and Edward Meeds. Swan: Sgd with normalization and whitening
350 enables stateless llm training. *arXiv preprint arXiv:2412.13148*, 2024.
- 351 [25] James Martens and Roger Grosse. Optimizing neural networks with Kronecker-factored approximate
352 curvature. In *ICML*, 2015.
- 353 [26] Depen Morwani, Itai Shapira, Nikhil Vyas, Eran Malach, Sham Kakade, and Lucas Janson. A new
354 perspective on Shampoo’s preconditioner. *arXiv preprint arXiv:2406.17748*, 2024.
- 355 [27] Guilherme Penedo, Hynek Kydlíček, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra,
356 Thomas Wolf, et al. The fineweb datasets: Decanting the web for the finest text data at scale. *Advances in*
357 *Neural Information Processing Systems*, 37:30811–30849, 2024.
- 358 [28] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language
359 models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- 360 [29] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In
361 *ICML*, 2018.
- 362 [30] Hao-Jun Michael Shi, Tsung-Hsien Lee, Shintaro Iwasaki, Jose Gallego-Posada, Zhijing Li, Kaushik
363 Rangadurai, Dheevatsa Mudigere, and Michael Rabbat. A distributed data-parallel PyTorch implementation
364 of the distributed Shampoo optimizer for training neural networks at scale. *arXiv preprint arXiv:2309.06497*,
365 2023.
- 366 [31] Chongjie Si, Debing Zhang, and Wei Shen. Adamuon: Adaptive muon optimizer. *arXiv preprint*
367 *arXiv:2507.11005*, 2025.
- 368 [32] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced
369 transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- 370 [33] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas
371 Janson, and Sham Kakade. SOAP: Improving and stabilizing Shampoo using Adam. In *ICLR*, 2025.
- 372 [34] Ge Yang, Edward Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub
373 Pachocki, Weizhu Chen, and Jianfeng Gao. Tuning large neural networks via zero-shot hyperparameter
374 transfer. *Advances in Neural Information Processing Systems*, 34:17084–17097, 2021.
- 375 [35] Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Diederik P Kingma, Yinyu Ye,
376 Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use fewer learning rates to gain more. In *ICLR*, 2025.
- 377 [36] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian.
378 GaLore: Memory-efficient LLM training by gradient low-rank projection. In *ICML*, 2024.
- 379 [37] Hanqing Zhu, Zhenyu Zhang, Wenyan Cong, Xi Liu, Sem Park, Vikas Chandra, Bo Long, David Z Pan,
380 Zhangyang Wang, and Jinwon Lee. APOLLO: SGD-like memory, AdamW-level performance. In *MLSys*,
381 2025.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: Abstract and Section 1 state the contributions; experiments in Section 5 match the claimed scope (GPT-2-modern up to 760M).

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 discusses the residual $\mathcal{O}(n^2)$ state overhead over AdamW and the fragility of the remaining eigendecomposition under data-parallel sharding.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Lemma 1 states its assumptions inline and is proved in Section 4.2.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 5.1 fixes the recipe and Algorithm 1 plus Section 4.2 fix the optimizer and default HPs.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: Code is not released at submission (release deferred to post-acceptance); the dataset is public and the algorithm is fully specified in Algorithm 1.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 5.1 specifies the LR/WD/ β grid, μ P transfer, seeds, and reporting convention.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Small-scale results are mean $\pm$ std over 3 seeds; medium and large are single-seed at the μ P-transferred LR, flagged as such in Section 5.1.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

430 Justification: $8 \times$ H100 (80GB), 1 node for small/medium and 2 nodes for large via FSDP;
431 per-step wall-time and state size are in Table 2.

432 **9. Code of ethics**

433 Question: Does the research conducted in the paper conform, in every respect, with the
434 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

435 Answer: [Yes]

436 Justification: Foundational optimizer research on the public FineWeb [27] corpus; no human
437 subjects, anonymity preserved.

438 **10. Broader impacts**

439 Question: Does the paper discuss both potential positive societal impacts and negative
440 societal impacts of the work performed?

441 Answer: [N/A]

442 Justification: Generic optimizer with no direct path to negative applications beyond the
443 application-agnostic effect of cheaper LLM training.

444 **11. Safeguards**

445 Question: Does the paper describe safeguards that have been put in place for responsible
446 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
447 image generators, or scraped datasets)?

448 Answer: [N/A]

449 Justification: No model checkpoints, image generators, or scraped data are released.

450 **12. Licenses for existing assets**

451 Question: Are the creators or original owners of assets (e.g., code, data, models), used in
452 the paper, properly credited and are the license and terms of use explicitly mentioned and
453 properly respected?

454 Answer: [Yes]

455 Justification: All baselines and ingredients are cited at first use; FineWeb [27] is used under
456 its public release terms.

457 **13. New assets**

458 Question: Are new assets introduced in the paper well documented and is the documentation
459 provided alongside the assets?

460 Answer: [N/A]

461 Justification: No new assets are released at submission time.

462 **14. Crowdsourcing and research with human subjects**

463 Question: For crowdsourcing experiments and research with human subjects, does the paper
464 include the full text of instructions given to participants and screenshots, if applicable, as
465 well as details about compensation (if any)?

466 Answer: [N/A]

467 Justification: No crowdsourcing or human subjects.

468 **15. Institutional review board (IRB) approvals or equivalent for research with human
469 subjects**

470 Question: Does the paper describe potential risks incurred by study participants, whether
471 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
472 approvals (or an equivalent approval/review based on the requirements of your country or
473 institution) were obtained?

474 Answer: [N/A]

475 Justification: No human subjects research.

476 **16. Declaration of LLM usage**

477 Question: Does the paper describe the usage of LLMs if it is an important, original, or
478 non-standard component of the core methods in this research? Note that if the LLM is used
479 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
480 scientific rigor, or originality of the research, declaration is not required.

481 Answer: [N/A]

482 Justification: LLMs are the subject of training, not a component of the method; any LLM
483 use during writing was limited to editing.